

ScholarPath

Technical Architecture & Security Brief

A walkthrough of the architecture, the request pipeline (CQRS), data integrity, and the security-critical features — encryption, file uploads & scanning, PII handling, live meetings, AI/RAG, and the booking–payment money flow. Every mechanism below is taken from the live codebase, not a plan.

Scope: .NET 10 Clean-Architecture monolith · React 19 SPA · Azure · ≈ 55 entities · ≈ 33 ports / ≈ 46 adapters · bilingual EN/AR.

Contents

1	Architecture — the foundation	1
2	CQRS & the request pipeline	1
2.1	The behavior pipeline (order matters)	1
3	Data integrity & “non-appearance” (soft delete)	2
3.1	Soft delete — how a row “disappears” without being destroyed	2
3.2	The <code>SaveChangesAsync</code> interceptor (one place, many guarantees)	2
3.3	Other integrity controls	2
4	The security model (defense in depth)	3
5	Encryption at rest (AES-256-GCM)	3
5.1	Key management	4
6	Authentication & authorization	4
7	File upload pipeline	4
8	Malware scanning (ClamAV, fail-closed)	4
9	PII handling	5
10	Live meetings (Azure Communication Services)	5
11	The AI subsystem	5
12	RAG — Retrieval-Augmented Generation	6
13	Booking & the payment money flow	6
13.1	Lifecycle (step by step)	6
13.2	Profit-share split	7
13.3	Webhook reconciliation	7
14	Cross-cutting engineering (quick hits)	7

1. Architecture — the foundation

ScholarPath is a **Clean-Architecture** monolith. Dependencies always point *inward*; the inner layers know nothing about the outer ones.

API → Application → Domain Infrastructure → (implements Application & Domain ports)

- **Domain** — entities, enums, domain events, and two domain ports (`ICurrentUserService`, `IDateTimeService`). No framework dependency.
- **Application** — use-cases as **CQRS** commands/queries (`MediatR`), `FluentValidation` validators, `AutoMapper` profiles, the AI cost gate, and the ≈ 33 **ports** (interfaces) the outside world must satisfy.
- **Infrastructure** — the ≈ 46 **adapters** implementing those ports: EF Core, Stripe, Azure OpenAI, ACS, Blob, Key Vault, Event Hub, SignalR, Hangfire jobs.
- **API** — controllers + Stripe webhook receiver, ASP.NET Identity + JWT, three SignalR hubs.

Why we did it this way

The *ports & adapters* seam is the most important decision in the system. It makes the code (a) **testable** — swap a real service for a stub; (b) **portable** — swap a provider by *configuration*, with no code change (`AI:Provider`, `Storage:Provider`, `Email:Provider`, `AcS:ConnectionString`, `FileScanning:Enabled`, `Hangfire:Enabled`); and (c) keeps business rules independent of Azure.

2. CQRS & the request pipeline

Every use-case is a **command** (writes/mutations) or a **query** (reads), dispatched through **MediatR**. This is CQRS: the read path and the write path are separate request types with separate handlers and validators.

2.1 The behavior pipeline (order matters)

Every request passes through four *pipeline behaviors* before reaching its handler. They are registered — and therefore execute — in this exact order (outermost first):

```
cfg.AddOpenBehavior(typeof(LoggingBehavior<, >>)); // 1 outermost
cfg.AddOpenBehavior(typeof(ValidationBehavior<, >>)); // 2
cfg.AddOpenBehavior(typeof(PerformanceBehavior<, >>)); // 3
cfg.AddOpenBehavior(typeof(AuditBehavior<, >>)); // 4 closest to handler
```

1. **Logging** (outermost) — wraps everything, so it records the request even if validation rejects it.
2. **Validation** — runs all `FluentValidation` validators for the request and short-circuits with a 400 before any work happens. Placed early on purpose: no point timing or auditing a request that is structurally invalid.
3. **Performance** — times the handler and flags slow requests.
4. **Audit** (innermost) — records the actual business action right around the handler, where the request is known-valid.

Why we did it this way

Ordering is deliberate: **Logging** must see every request; **Validation** must reject bad input *before* we spend time or write audit rows; **Audit** must sit closest to the handler so it only records actions that actually executed. Validators, `AutoMapper` profiles, and the refund calculator are auto-discovered from the assembly, so adding a feature needs no DI wiring.

3. Data integrity & “non-appearance” (soft delete)

3.1 Soft delete — how a row “disappears” without being destroyed

Sensitive/business rows are never hard-deleted. They implement `ISoftDeletable` (`IsDeleted`, `DeletedAt`, `DeletedByUserId`); “deleting” flips `IsDeleted = true`. To make those rows vanish from the whole application automatically, each such entity has an EF Core **global query filter**:

```
b.HasQueryFilter(x => !x.IsDeleted); // applied on ~15 entities
```

How it works

Once the filter is on an entity, *every* LINQ query EF generates for it gets an implicit `WHERE IsDeleted = 0`. A developer cannot forget to add it — soft-deleted rows simply never appear in any query, join, or count, unless code explicitly opts out with `IgnoreQueryFilters()` (used only by admin/GDPR tooling). 15 entities carry this filter today.

Why we did it this way

Hard deletes would cascade-break history (reviews, payments, audit trails) and make “who deleted what, when” impossible. Soft delete keeps referential history intact, supports an undo/restore path, and — combined with the audit log — gives full traceability. It is also the safe default for a platform that must honor data-retention and dispute rules.

3.2 The `SaveChangesAsync` interceptor (one place, many guarantees)

The `DbContext` overrides `SaveChangesAsync` to enforce four cross-cutting rules in a single choke point:

- **Audit stamping** — sets `CreatedAt` on new auditable rows and `UpdatedAt` on every modification (seeders keep intentional back-dates).
- **Domain events after commit** — domain events are dispatched *only after* the transaction is durably persisted, so a side effect never fires for a change that rolled back.
- **Conflict translation** — a unique-index violation (SQL 2601/2627) is caught and re-thrown as a friendly `ConflictException` (e.g. “that slot was just booked”) instead of a raw 500.
- **Global cascade safety** — every FK pointing at `ApplicationUser` is swept to `DeleteBehavior.Restrict`, avoiding SQL Server’s multiple-cascade-path error and protecting user history.

3.3 Other integrity controls

- **Optimistic concurrency** — `RowVersion` on auditable entities prevents lost updates.
- **Idempotency keys** — `Payment.IdempotencyKey` and `Notification.IdempotencyKey` make money and messaging exactly-once under retries; Stripe webhooks dedupe on `StripeWebhookEvent.StripeEventId`.
- **Filtered unique indexes** encode real invariants in the database: one *active* application per student per scholarship (FR-057), one *active* booking per consultant per slot, one *active* financial config per type.

4. The security model (defense in depth)

Independent, layered controls so no single failure is catastrophic:

Layer	Control
Transport	HTTPS everywhere; JWT bearer on the API; authenticated SignalR hubs
Identity	ASP.NET Core Identity — hashed passwords, logout, email confirmation
Tokens	Short-lived access JWT + <i>rotating</i> refresh tokens, stored <i>hashed</i>
Authorization	Role-based (Student/Company/Consultant/Admin) + per-resource ownership

Secrets & keys	Azure Key Vault key providers (field-encryption key, JWT signing key)
Data at rest	AES-256-GCM field encryption on sensitive columns
Uploads	Scan-before-store with ClamAV, fail-closed
Privacy / PII	Prompt redaction before AI; loose references; GDPR export/delete
Integrity	Idempotency keys, filtered unique indexes, RowVersion
Auditability	Append-only AuditLog (before/after JSON); per-call AiInteraction log
Soft delete	Global query filters hide deleted rows from every query

5. Encryption at rest (AES-256-GCM)

Port `IFieldEncryptionService` → `AesGcmFieldEncryptionService`, wired onto chosen PII columns (e.g. a profile Biography, an application's PersonalNotes) through an EF Core value converter — plaintext in C#, ciphertext in the database.

How it works

AES-256-GCM is an *authenticated* (AEAD) cipher: 256-bit key, a fresh random 12-byte nonce per encryption, and a 16-byte authentication tag. The stored value is a self-describing envelope `enc:v1: + Base64(nonce || tag || ciphertext)`. On decrypt the tag is verified — a flipped bit, a truncation, or the wrong key makes decryption *throw*, not return garbage. The random nonce means the same plaintext encrypts to different ciphertext each time.

- **Versioned** (v1): a future key rotation introduces v2 while still decrypting old v1 rows.
- **Safe rollout**: a value without the prefix (legacy plaintext) is returned unchanged and re-encrypted on its next write — zero downtime.
- **Key** comes from `IFieldEncryptionKeyProvider` (Key Vault in prod, local in dev) and must be exactly 32 bytes (checked at start-up).

Security rationale

GCM gives confidentiality *and* integrity in one pass; a plain cipher would protect secrecy but not detect tampering. The key never lives in source or config. This protects the value end-to-end — even a DB backup or a DBA sees only ciphertext for those columns (which transparent DB-level encryption alone cannot guarantee once a query returns a row).

5.1 Key management

Two independent providers, each with a prod + dev adapter: `IFieldEncryptionKeyProvider` (KeyVault / Local) for the AES key, and `IJwtKeyProvider` (KeyVault / Local-PEM) for the JWT signing key. Surfacing the signing key behind its own port is what makes **key rotation** an operations action, not a redeploy.

6. Authentication & authorization

- **ASP.NET Core Identity** — `ApplicationUser : IdentityUser<Guid>`; passwords hashed by `IdentityPasswordHasher`; logout; email confirmation.

- **Tokens** — `TokenService` issues a short-lived access JWT plus a refresh token; refresh tokens are *rotated* on use, individually/globally revocable, and stored *hashed* (`RefreshToken.TokenHash`). Same hashing for `PasswordResetToken` — a DB leak yields no usable token.
- **SSO** — `SsoService` exchanges OAuth codes with Google and Microsoft.
- **Authorization** — role-based (the overlapping Student/Company/ Consultant/Admin specialization via the `UserRoles` join) plus per-resource ownership checks in handlers.

7. File upload pipeline

Port `IBlobStorageService` → `FileStorageService`: one interface, two providers chosen by `Storage:Provider` — `Local` (disk, dev) or `AzureBlob` (Azure Storage). Objects use an opaque, provider-tagged path `provider:container/key`, where `key` is GUID-prefixed so same-named uploads never collide; Azure containers are created `PublicAccessType.None` — never public, downloads always go through the authenticated API.

How it works

Scan before store. The upload handler runs the file through the malware scanner *while it is still an in-memory stream; only a Clean verdict is persisted*. This guards the document vault (application attachments, upgrade files, forum attachments, session recordings).

8. Malware scanning (ClamAV, fail-closed)

Port `IFileScanService` → `ClamAvFileScanService`: bytes are streamed to a ClamAV `clamd` daemon over `INSTREAM` (via `nClam`) before storage. Verdicts: **Clean** (proceed; stream rewind so the same bytes are stored), **Infected** (rejected; signature logged), **ScanUnavailable** (rejected).

Security rationale

Fail-closed. Any daemon unreachability, timeout, or error maps to `ScanUnavailable` and the upload is **rejected** — an unscanned file is never stored. (Feature-flagged by `FileScanning:Enabled`; `NoOpFileScanService` in dev.) Scanning the stream first, fail-closed, removes the window in which a malicious file could be reachable.

9. PII handling

PII protection is layered across the AI path, the data model, and the lifecycle.

How it works

Prompt redaction before persistence *and* before any provider. In the chatbot handler the user's message passes through `RedactPii()` first: emails → [redacted-email], 13–19-digit runs (cards) → [redacted-card], 10+-digit runs (phones) → [redacted-phone]. The *redacted* text is what we store (`AiInteraction.PromptText`) and what we send to OpenAI — the raw message never leaves the process. (Regex order matters: email, then card, then the most permissive phone pattern last.)

- **Personalization without contact PII** — the student-profile block injected for tailored answers carries only academic facts (nationality, level, field, GPA, preferred countries), never email/phone.
- **Per-user AI memory** — replayed history is filtered to the requesting user, so a leaked `SessionId` cannot leak another user's transcript.
- **Redaction QA** — `AiRedactionAuditSample` stores periodic samples of redacted prompts for human review (a sampling job) — we can prove the redactor works.
- **Loose references** — high-volume / audit / analytics tables (payments, notifications, votes, chat, audit log, AI interactions) hold the user `Guid` with *no FK constraint*, so a user can be

deleted or anonymized *without* cascade-breaking history.

- **GDPR** — `UserDataRequest` drives export and delete (`DataExportJob`, `DataDeleteJob`); sensitive free-text is encrypted at rest; profile fields are role-gated in the UI.

10. Live meetings (Azure Communication Services)

Port `IMeetingService` → `AzureCommunicationMeetingService` (selected when `Acs:ConnectionString` is set; `StubMeetingService` in dev).

How it works

A booking's "room" is an **ACS group-call id (a GUID)** — no server-side room object. On join, the API mints a **short-lived, VoIP-scoped access token per user** from ACS Identity (`CreateUserAndTokenAsync`, with an expiry), handed only to the two booking participants through the authorized join endpoint. **Recording** uses ACS Call Automation (`StartRecordingAsync` → `RecordingId`; `DownloadRecordingAsync` streams it), persisted as a `SessionRecording` + Blob object.

Attendance & no-shows: the booking records `RecordingStartedAt` (PB-006) and `StudentJoinedAt` / `ConsentAt` (FR-217); the `MeetingNoShowSweepJob` marks `NoShowMarkedAt` and drives the no-show/refund logic.

Security rationale

A video token is a bearer credential. Minting one *per participant*, scoped to VoIP and expiring, means a leaked token is useless after the session and cannot be replayed for another call.

11. The AI subsystem

Port `IAIService` has **three interchangeable adapters** chosen by `Ai:Provider`: `AzureOpenAiService`, `OpenAiService` (OpenAI direct), `LocalAiService` (deterministic, offline). Embeddings mirror this. Three operations: `Recommend()`, `CheckEligibility()`, `Ask()`.

- **Recommendations are not an LLM call** — they are produced by the deterministic `LocalAiService`, scoring scholarships against the student profile + metadata (+ RAG), returning the top-N (default 5). The LLM backs *chat* / *Q&A*.
- **Chat (Ask)** carries session memory (last 20 turns, user-scoped), an injected profile system-turn for personalization, RAG grounding, and returns the answer + a disclaimer + the *sources* used.

Security rationale

Cost control — AiCostGate. Before *every* AI call we enforce a rolling 24-hour per-user budget (`DailyUserCostLimitUsd`, default \$1.00): we sum the user's `AiInteraction.CostUsd` over 24h and reject the call if this one would exceed the cap. Every call is logged (provider, model, tokens, cost, and a metadata blob of RAG sources) — full spend observability. What keeps the bill bounded: the per-user gate *and* the fact that the high-frequency feature (recommendations) is local, not an LLM call.

12. RAG — Retrieval-Augmented Generation

How it works

Index — `KnowledgeBaseIndexer` turns scholarships, resources, consultants and top community posts into `KnowledgeDocument` rows (bilingual content, a `ContentHash`, and an embedding `byte[]` tagged with the model name + dimensions; `SourceType+SourceKey` unique, so re-indexing upserts). **Retrieve** — `KnowledgeRetriever` embeds the query, ranks the corpus by

cosine similarity and returns the **top-K**, considering only documents embedded with the *currently active* model (vectors from another model live in a different space).

Why we did it this way

The corpus is a few hundred rows, so an in-memory similarity scan per query is more than fast enough — we deliberately avoid operating a separate vector database. Which documents grounded each answer is recorded in `AiInteraction.MetadataJson` and surfaced as “sources”, giving a RAG audit trail and reducing hallucination.

13. Booking & the payment money flow

Paid consultant sessions use an **authorize-then-capture (escrow-style)** model on Stripe, so the student’s money is *held* but not taken until the consultant accepts. Port `IStripeService` → `StripeService` (real) / `StubStripeService` (dev).

13.1 Lifecycle (step by step)

1. **Book & authorize** — the student requests a slot; we create a Stripe `PaymentIntent` with **manual capture** and `PaymentMethodTypes = [card]`. The card is *authorized* (funds held), not charged. A `Payment` row is created with an `IdempotencyKey` (the same key is sent to Stripe, so a retry never double-charges). Amounts are integer **cents**.
2. **Accept** → **capture** — when the consultant accepts, we **capture** the held intent (`CapturePaymentIntentAsync`). A capture on an un-authorized intent is surfaced as a clean 422 (“finish payment first”), not a 500.
3. **Reject / expire** → **release** — the hold is cancelled (`CancelPaymentIntentAsync`); the student is never charged.
4. **No-show / cancellation** → **refund** — a `RefundCalculatorService` computes the owed amount per policy, then `RefundPaymentAsync` issues a full/partial refund. `MeetingNoShowSweepJob` triggers this automatically.
5. **Payout to consultant** — consultants onboard a **Stripe Connect Express** account (`CreateConnectAccount` + onboarding link). The `StripePayoutJob` pays out their net via `CreatePayoutAsync` (idempotent), recording a `Payout` row that aggregates the included payment ids.

13.2 Profit-share split

Each `Payment` stores `AmountCents`, `ProfitShareAmountCents` (platform fee) and `PayeeAmountCents` (consultant net). The split percentages come from `ProfitShareConfig` / `FinancialConfigRule`, each protected by a **filtered unique index** that enforces exactly one *active* rule per payment type (PB-014 / FR-170) — so fees are configurable and versioned over time without ambiguity.

13.3 Webhook reconciliation

`StripeService.ParseWebhook` **verifies the Stripe signature** and extracts the typed event (`PaymentIntent`, `Charge`, `Account`, `Payout`, `Dispute`). The handler reconciles the matching `Payment` by intent id, and dedupes on `StripeWebhookEvent.StripeEventId` so a re-delivered event is processed once.

Security rationale

Three independent guarantees protect the money path: **idempotency keys** (no double-charge / double-payout on retries), **manual capture** (funds are only taken when the service is actually committed), and **signed webhooks + event dedupe** (no spoofed or replayed payment events). Card details never touch our servers — the browser talks to Stripe directly via the

client secret; we only ever see the intent id.

Likely question

“Where is the money between booking and the session?” — It is *authorized and held* on the student’s card via a manual-capture `PaymentIntent`. We capture it only when the consultant accepts; if they reject or the slot expires we release the hold; if there’s a qualifying no-show/cancellation the `RefundCalculatorService` decides the refund. Net earnings reach the consultant through a Stripe Connect payout.

14. Cross-cutting engineering (quick hits)

- **Audit log** — `AuditService` writes append-only `AuditLog` rows (before/after JSON, polymorphic target).
- **Real-time** — three SignalR hubs (Chat, Notification, Community).
- **Background jobs** — 13 Hangfire jobs (reminders, scholarship auto-close, no-show sweep, payouts, GDPR export/delete, redaction sampling, integrity checks), *feature-flagged* via `Hangfire:Enabled`.
- **Bilingual EN/AR** with RTL throughout.
- **Documentation rigor** — EERD, Relational-Mapping, Class and Component reports, each gap-analyzed against the live code and corrected.

All mechanisms above are implemented in `server/src/...` and verified against the source for this brief.